UCAG — University Course Application Guide

Project handover document — last verified 15 October 2026 Purpose: a complete, current, and independently-verified picture of this project for anyone (human or AI) picking up development work on it — what it is, how it's built, what's actually live versus what only exists in code, and what to do next.

Live site: **https://ucag-nine.vercel.app** Repository: `saxs-14/UCAG` on GitHub, deployed via Vercel's Git integration (`saxs-14s-projects/ucag`)

---

1. What this project is

UCAG helps South African Grade 12 learners answer one question with confidence: **"What can I actually study, and where?"**

A learner enters their NSC (National Senior Certificate) subjects and marks once. From that, the app:

- Calculates their **APS (Admission Point Score)** correctly **per institution** — not a single generic number, because every South African university converts raw marks into a score differently (some exclude Life Orientation, some weight it, some use raw percentages instead of point bands, some apply bonus points for specific subjects). There is no one "APS formula."
- Shows which programmes they **qualify** for, which they're **almost qualified** for (with the exact point or subject gap), and which they **don't yet** qualify for — with a realistic alternative pathway instead of a dead end.
- Shows real **application windows** (open/opening-soon/closed) with the correct call to action — an apply link when applications are genuinely open, a status-check link when closed, never a dead or misleading link.
- Surfaces matched **bursaries and internships**, with an explicit scam-warning pattern (upfront-fee requests, no verifiable provider, social-media-only sourcing) auto-flagged and never rendered.
- Lets a signed-in learner save their marks, shortlist programmes, and track their own application-readiness checklist.
- Offers an AI chat assistant (Google Gemini-backed) that is explicitly scoped to **general guidance only** — it is grounded in the app's own verified Firestore data and is instructed to never state an institution-specific requirement or date itself; those only ever come from the calculator's sourced results.

The one rule that governs every decision in this codebase

> **Unverified is displayed as unverified. Never as a fact.**

This is a fact-delivery product for a decision a learner gets to make once a year. A wrong closing date or a fabricated APS rule costs someone a real year of their life. Every fact-bearing record in the database

carries three fields — `sourceUrl`, `verifiedOn`, `academicYear` — and a document missing any one of them **does not render**, enforced by a shared type guard (`isFactVerified()` in `lib/firestore/types.ts`), not by convention or code review discipline. When the pipeline can't determine something, the UI says "not yet verified" and links to the institution directly — it never guesses.

This rule is why large parts of the UI deliberately render "data pending verification" rather than invented numbers — that is correct behavior, not an unfinished feature.

---

2. Tech stack

Layer	Choice
Framework	Next.js 15 (App Router), TypeScript strict mode
Styling	Tailwind CSS v4, design tokens as CSS custom properties in `app/globals.css`
Database	Firestore (Google Cloud project `ucag-11adc`)
Auth	Firebase Authentication (email/password + Google, plus anonymous sessions for the calculator)
AI / LLM	Google Gemini (`LLM_PROVIDER=gemini`), free-tier `gemini-flash-lite-latest` by default — used for both the ingestion pipeline's data extraction and the chat assistant
Scheduled jobs	Vercel Cron (Hobby plan caps this at once/day)
Validation	Zod on every AI output and every form
Charts	Recharts, lazy-loaded and skipped entirely in low-data mode
Testing	Vitest (unit), Playwright (E2E), Firebase Emulator Suite (real Firestore/Auth rules testing, not mocked)
Hosting	Vercel (Hobby plan), Git-linked to `master`
CI	GitHub Actions

Hard architectural rule: `lib/aps/` (the APS calculation engine) is pure, dependency-free TypeScript — no Firebase, no React, not even an import from another `lib/` or `config/` directory. This keeps it unit-testable in total isolation and portable to a different runtime (e.g. a Flutter/Dart port, or a standalone serverless function) without dragging the rest of the app along.

---

3. Architecture

3.1 The APS engine (`lib/aps/engine.ts`)

One implementation handles every real South African formula variant seen so far, selected by configuration rather than by branching code per institution:

- **Point-band sum** (most common — raw percentage converted to a 1–7 point band, best N subjects summed)
- **Point-band with bonus** (e.g. Wits: best 7 including Life Orientation, plus bonus points for 60%+ in English/Maths)
- **Percentage sum** (e.g. Stellenbosch's general formula — raw percentages added directly, no point-band conversion)
- **Faculty point score** (e.g. UCT, Stellenbosch Engineering/Science — a faculty-specific weighted formula, different from the institution's general rule)
- **Applicant-score-weighted** formulas with bonus/equity rules

Configurable per rule: which subjects are forced into the count regardless of ranking (e.g. UCT always counts English + Maths), which subjects are counted an *additional* time on top of their normal inclusion (e.g. Stellenbosch Engineering double-counts Maths and Physical Science), a divisor for genuine weighted-average formulas, Life Orientation treatment (exclude / include / half-weight / capped), Math Literacy policy, and bonus rules gated on real applicant context (never fired on a guess).

An institution can have **more than one** `ApsRule` — one institution-wide default (`facultyId: null`) plus faculty-specific overrides — resolved by `lib/matching/resolveApsRule.ts` , which prefers the faculty-specific rule when one exists.

3.2 The matching engine (`lib/matching/engine.ts`)

Buckets a learner into **qualify / almostQualify / notYet** against a specific programme's APS and subject requirements, itemizing every reason — met and unmet — and suggesting a concrete next step when not yet qualifying. A programme at an institution with **no verified APS rule on record** never gets a fabricated score; it renders separately as "real programme, score not yet available" (`UnscoredProgrammeCard`), with everything that *is* verified (name, requirements, source) shown honestly.

3.3 Firestore schema & trust model (`lib/firestore/types.ts`)

Core collections: `institutions` , `faculties` , `schools` , `programmes` , `apsRules` , `applicationWindows` , `bursaries` , `internships` , `statistics` , plus pipeline-internal collections (`sources` , `ingestionRuns` , `verificationQueue` , `linkHealthChecks`) and per-user `userProfiles` .

Every fact-bearing document extends `FactProvenance` (`sourceUrl` , `verifiedOn` , `academicYear`) and must pass `isFactVerified()` before any public page will render it. This is checked at the data-fetch layer (`lib/catalog/getRealCatalog.ts` and siblings), not left to individual components to remember.

Security: Firestore rules deny all public writes outright — every write goes through an Admin-SDK-backed API route (`/api/admin/*`) that independently re-verifies a real Firebase ID token carries the `role: admin` custom claim, not a client-settable Firestore field. A minor's profile write is rejected by the rules

engine itself unless it carries a recorded guardian-consent, regardless of what the client sends (POPIA compliance, enforced not just documented).

3.4 The AI ingestion pipeline (`lib/ingestion/`)

A source-fetch → extract → diff → route → verify pipeline, built honestly in stages rather than all at once:

- `robotsCheck` — actually parses and respects each source's robots.txt
- `budget` — a real, enforced token-spend kill switch, not just a configured limit
- `fetchSourceText` / `htmlToPlainText` — fetch with conditional HTTP (304 support), retries, content hashing
- `extract` — Zod-schema-validated LLM extraction; rejects and logs schema-invalid output rather than coercing it into something that looks right
- `diff` — compares extracted values against what's already on record
- `route` — decides auto-publish vs. human verification queue; **high-risk fields never auto-publish regardless of model confidence**
- `bursarySafety` — the scam-detection ruleset (upfront-fee keywords, no verifiable provider site, social-media-only sourcing), reused both at ingestion time and again at render time as defense in depth

Two concrete extraction tasks are fully wired end-to-end and have been live-tested against a real Gemini key: `applicationWindows` and `programmeRequirements`. Both always land in the admin **verification queue** — nothing from the AI pipeline is ever auto-published to a learner-facing page.

There is also a standalone trained model (`scripts/train-bursary-scam-model.mts`) for bursary-scam classification, separate from the rule-based `bursarySafety` checks.

3.5 Admin console (`/admin/*`)

Role-gated (`requireAdmin` , checked server-side on every write): a verification queue (approve/edit/reject each pending fact, one keystroke each), a source register, ingestion-run history with token/cost tracking, a generic JSON-patch content editor, and a dead-link report that really persists HTTP-check results to Firestore (not a stub).

3.6 Chat assistant (`app/api/chat/` , `lib/chat/`)

Server-Sent-Events streaming chat backed by Gemini, rate-limited (`lib/chat/rateLimiter.ts`), grounded in the app's own verified Firestore data via `lookupVerifiedFact` so it can say "yes, that's in our verified records" instead of guessing — and explicitly instructed to refuse institution-specific requirement/date claims, deferring those to the calculator's sourced results instead.

---

4. Full feature / route inventory

Route	What it does
`/`	The calculator — subject/marks entry, live APS + eligibility results, readiness checklist, APS improvement simulator, interest-based recommendations, course comparison
`/institutions`, `/institutions/[id]`	Institution directory and per-institution profile (campuses, APS formula summary, official links)
`/programmes`, `/programmes/[id]`	Full programme search/filter and individual programme detail pages (server-rendered, SEO'd with JSON-LD)
`/bursaries`	Bursary/internship discovery, filterable by field of study and level, with the scam-warning explainer rendered inline
`/ump/*`	A dedicated, richer University of Mpumalanga micro-site (campus life, careers, funding, learnerships) — UMP is the project's original, most-complete institution
`/login`, `/register`	Age gate → guardian-consent capture (under-18) → email/Google sign-up; separate sign-in form
`/account`	Signed-in dashboard — saved marks, shortlist, working "download my data" and "delete my account" (POPIA rights, both real, both live-tested against the emulator)
`/privacy`	Plain-language privacy notice
`/application/documents`	Generic application-readiness document checklist
`/admin/*`	Verification queue, source register, ingestion runs, content editor, dead-link report (role-gated)
`/api/chat`	Streaming AI chat assistant
`/api/cron/link-health`	Scheduled link-health check (Vercel Cron, `CRON_SECRET`-protected)
`/api/admin/*`	Admin-only write endpoints, independently re-verified server-side

---

5. ⚠️ Current live production status — the most important section in this document

This is the gap most worth understanding before doing any more feature work, because it is **not visible from reading the code or the commit history** — only from actually exercising the live site.

The codebase contains real, verified, sourced data for 9 institutions (University of Mpumalanga, University of Pretoria, University of the Witwatersrand, North-West University, Nelson Mandela University, University of KwaZulu-Natal, Tshwane University of Technology, Stellenbosch University, University of Cape

Town) — 138+ real programmes with real APS formulas, real subject requirements, and real source citations, each committed as its own `config/<institution>Programmes.seed.ts` file plus a `npm run seed:<institution>-data` script to write it to Firestore.

Verified directly against the live production site today (not assumed from the code):

- `/programmes` shows **"12 of 12 programmes shown"**, and the institution filter dropdown lists only **UMP**. None of the other 8 institutions' real programme catalogues have actually been written to production Firestore.
- `/bursaries` shows **one single bursary** (SAA Cadet Pilot Programme). The ~36 other real, sourced bursaries built and committed this project (Allan Gray Orbis, FirstRand Empowerment, Toyota SA Motors, NSFAS-adjacent listings, etc.) are not live either.
- Institution **profile** pages (e.g. `/institutions/stellenbosch`) *do* render correctly with real campus and APS-formula information — the `institutions` collection itself is seeded. It's specifically the `programmes` and `bursaries` collections where the gap is.

Why: writing to the real production Firestore requires real Firebase Admin credentials (a service-account key), which live only as a Vercel environment variable marked "Sensitive" (unretrievable via the Vercel CLI/API by design) or via a locally authenticated `gcloud` session. Scripted/automated attempts to seed production without those credentials present locally are correctly rejected rather than silently routed around. **Someone with real access to the Vercel project's environment variables or the Firebase service account needs to run the seed scripts themselves**, from a machine with that service account's credentials in `.env.local`.

The fix is mechanical, not a rebuild. With a real `.env.local` populated from the actual Firebase Admin service account (see `GOOGLE_APPLICATION_CREDENTIALS` / `FIREBASE_ADMIN_*` vars), run, in order:

```
npm run seed:institutions          # if institutions ever need re-seeding
npm run seed:aps-rules
npm run seed:up-data
npm run seed:wits-data
npm run seed:nwu-data
npm run seed:nmu-data
npm run seed:ukzn-data
npm run seed:tut-data
npm run seed:stellenbosch-data
npm run seed:uct-data
npm run seed:bursaries-statistics
```

Each script is idempotent (keyed by document ID) and only writes the specific institution's real, already-committed, already-cited data — there is no data-quality work left here, only the act of actually running the write.

Everything else reported as "live" or "working" elsewhere in this document has been independently re-verified (production route sweep, GitHub Actions status, live DOM content) as part of

writing this document, not assumed from prior documentation.

---

6. CI/CD — keeping GitHub and Vercel green

GitHub Actions (`.github/workflows/`)

The main workflow ("UCAG CI & Deployment Preview Check") runs on every push to `master` : typecheck (`tsc --noEmit`), lint (ESLint), a **real Firestore emulator** spun up for the test run (not mocked), the full Vitest suite, a production build (`next build --turbopack`), and a calculator-route bundle-budget check (currently ~257KB against a 260KB budget — tight; watch this on the next change that touches the calculator route or its dependencies).

A separate scheduled workflow (`cron-uptime-check.yml`) pings the production cron endpoint every 6 hours and fails loudly on a non-2xx response.

Status as of this document: both green. Last code-change CI run and the most recent scheduled uptime check both succeeded.

To keep it green on every future change, before pushing:

```
npm run typecheck
npm run lint
npm test -- --run
npm run build
npm run check:bundle-budget -- <path-to-build-output-log>
```

If a PR or push breaks CI, read the actual GitHub Actions log (`gh run view <run-id> --log-failed`) rather than guessing — the four checks above reproduce the CI job exactly, so a local failure is the same failure CI will report.

Vercel

Git-linked to `master` on the Hobby plan (`saxs-14s-projects/ucag`). A plain `vercel` or a `git push` to `master` deploys straight to **production** — there is no separate staging environment configured, so anything not meant to go live needs `vercel --target=preview` explicitly. Vercel Hobby caps cron frequency at once/day, which is why `vercel.json` 's cron entry runs daily rather than the ingestion pipeline's ideal 6-hourly cadence (a Pro-plan upgrade would unblock that).

To verify Vercel is actually serving the latest deploy (the alias domain can occasionally serve a stale cached static asset — this bit the logo redesign once; see §9):

```
vercel ls --scope saxs-14s-projects       # confirm the newest deployment is "Ready" and "Pr
curl -s -o /dev/null -w "%{http_code}\n" https://ucag-nine.vercel.app/<route>   # sweep key
```

For any static asset (an icon, a JSON file, anything in `public/` or an App Router file-convention route like `app/icon.svg`), diff its actual byte content against what was just deployed — a 200 status code alone does not prove the content is current, as the favicon incident in §9 demonstrated.

---

7. Design system

- **Palette**: paper/ink neutrals plus a semantic "marking-pen" palette — green = qualify, gold = "almost, here's the exact gap," slate = not yet, red held in reserve for genuine errors only. Every token is a single CSS custom property in `app/globals.css` that flips automatically under `prefers-color-scheme: dark` — components reference `text-mark-green` once and it's correct in both themes.
- **Typography**: system-UI sans-serif stack for body text; a dedicated monospace stack (`font-mono tabular-nums`) specifically for numbers — APS scores, percentages, dates — so figures never jump around as they update.
- **Signature element**: `components/CircledMark.tsx` — a hand-drawn double-ellipse ring around a number, echoing a teacher circling a total on a marked exam script. This is the product's actual visual metaphor (see "The Marked Script" design direction).
- **Logomark**: a flat, geometric mortarboard (graduation cap) icon — navy badge, teal cap, gold tassel — deliberately simple (no 3D bevels, no literal pillars/books/starburst) so it reads as a clean product mark rather than stock clip art. Interactive: the tassel's weighted end breathes gently at rest, and the whole cap tips on hover/focus. Defined once in `components/Logo.tsx` and propagated to `app/icon.svg`, `app/favicon.ico`, and every PWA/social-share asset in `public/`.
- **Icons**: a small hand-authored inline SVG set (`components/icons/Icon.tsx`) — no icon-library dependency, kept deliberately minimal to protect the calculator route's bundle budget.
- **Low-data mode**: `lib/useSaveData.ts` reads `navigator.connection.saveData` and, when on, skips chart rendering (falls back to a plain data table) and any non-essential web-font loading entirely.
- **Accessibility**: global visible focus rings, `prefers-reduced-motion` honored everywhere, a skip-to-content link, 44px minimum touch targets. Lighthouse (Accessibility/SEO/Best Practices) scored 100/100/100 on the last audit; no screen-reader or automated axe-core audit has been done independently of Lighthouse — flagged as a real gap, not claimed as covered.

---

8. Repository map (where to find things)

```
app/                    Next.js App Router pages and API routes
components/             React components, organized by feature area
  icons/                 Hand-authored inline SVG icon set
  aps/, results/, ...     Feature-specific component groups
lib/
  aps/                   Pure APS calculation engine (zero external deps)
  matching/               Qualify/almost/not-yet bucketing logic
  ingestion/              AI data-ingestion pipeline (fetch/extract/diff/route)
  chat/                   Chat assistant backend
  firestore/types.ts      The full data model + isFactVerified() trust gate
  catalog/                Client-facing data-fetch layer (Firestore -> typed data)
  admin/                  Server-side admin auth checks
config/                 Seed data: institutions, programmes per institution, subjects,
                         APS rules, bursaries, sources — all with real sourceUrl citations
scripts/                One-off and seed scripts (seed:*, admin:grant, bundle-budget check)
tests/                  Firestore rules tests, auth integration tests, E2E (Playwright)
docs/                   Historical planning/audit documents (see note below) +
                        this handover document
public/                 Static assets — icons, manifest.json, service worker, og-image
```

Note on `docs/`: this directory contains many earlier planning, audit, and status documents (`MASTER_PROMPT_v2.md`, `PROJECT_STATUS.md`, several `*_AUDIT.md` files, etc.) written at different points across the project's history. Some predate work described in this document and are now stale — in particular, any document claiming "no real Firebase project connected" is out of date (a real project, `ucag-11adc`, has been connected for some time). **Treat `CLAUDE.md` at the repo root and this document as the current source of truth**; reconcile anything else against the live app and git history rather than trusting an older doc at face value.

---

9. Notable incidents worth knowing about (so they aren't repeated)

- **Firestore document-ID collision**: an early APS-rules seed script keyed documents by `institutionId` alone; once an institution gained a second (faculty-specific) rule, the later write silently overwrote the first. Fixed by keying on `institutionId-facultyId` when a faculty ID is set. Lesson: always verify a seed script's actual write count in Firestore matches the expected count, don't just check the script exits cleanly.
- **`/bursaries` 500 crash**: one data-fetch function lacked the same try/catch-and-degrade-gracefully pattern every sibling fetcher used, so any transient Firestore/credentials issue crashed the whole page instead of rendering an honest empty state. Fixed; worth checking any *new* data-fetch function added later follows the same pattern.
- **A logo redesign shipped correctly to `public/icon.svg` but the site kept showing the old one.** Root cause: Next.js's App Router has its own file-convention favicon handling — `app/icon.svg` (a

separate file) takes priority over `public/icon.svg` and was never updated. General lesson for this codebase: when an asset has both an `app/` file-convention version and a `public/` version, they are **not** the same file, and only the `app/` one is actually served at that path.

- **Vercel's CLI deploy can land on production unintentionally.** `vercel` with no flags on a Git-linked project checked out on its production branch deploys straight to production — always pass `--target=preview` for anything not meant to go live immediately.

---

10. How to run and deploy

Local development (no real Firebase credentials needed):

```
firebase emulators:start --only firestore,auth --project demo-ucag
# in a second terminal, with NEXT_PUBLIC_USE_FIREBASE_EMULATOR=true in .env.local:
npm run dev
```

Full verification before any push:

```
npm run typecheck && npm run lint && npm test -- --run && npm run build
```

Granting an admin role (trusted-operator action, not self-service):

```
npm run admin:grant -- someone@example.com
```

Deploying: push to `master` (auto-deploys to production via Vercel's Git integration), or `vercel --target=preview` for a non-production preview build first.

---

11. Suggestions for what to prioritize next

In rough priority order, as an outside assessment of where effort would have the most real impact:

1. **Seed production Firestore with the real data already built and committed** (§5). This is by a wide margin the highest-leverage item on this list — the product's core value proposition (accurate eligibility across many institutions) is currently live for exactly one institution out of nine the codebase actually supports. Everything else below matters less until this is done.
2. **Add a CI check that would have caught the favicon incident**: a simple post-deploy smoke test that diffs the byte size or a content hash of a few known static assets (favicon, og-image, manifest) against what was just built, rather than only checking HTTP status codes.
3. **Close the remaining institutions**: UJ has a real, sourced APS rule but its programme catalogue was blocked by a 403 on the source site and was never built; CPUT and UNISA were deliberately

left unseeded (no verified source research done yet). Both are real, scoped next-institution work, following the same pattern already proven nine times over.

4. **Independent accessibility audit**: run axe-core or a real screen reader through the calculator and results flow once, beyond Lighthouse's own (less rigorous) accessibility check.
5. **Bundle budget has almost no headroom left** (~257KB of a 260KB budget on the calculator route). The next feature that touches that route should budget-check itself before merging, not discover the overage in CI.
6. **Wire the remaining ingestion tasks** (faculty/school structure, bursaries/internships, national statistics) the same way `applicationWindows` and `programmeRequirements` were — the orchestrator pattern (`lib/ingestion/*Pipeline.ts`) is proven and reusable.
7. **Reconcile or archive the stale docs in `docs/`** (§8) so a future reader — human or AI — doesn't act on outdated claims the way an earlier pass in this project nearly did.
8. **A real paid error-monitoring/uptime service** (Sentry or equivalent) once there's a budget for it — `lib/errorReporting.ts` already has the one-line integration point ready; right now errors only reach console logs, which Vercel captures but nobody is actively watching.

---

12. Quick facts reference

- **Owner**: Phathutshedzo "Saxs" Mamagau — SaxsProjects
- **Firebase project**: `ucag-11adc`
- **Vercel project**: `saxs-14s-projects/ucag` (Hobby plan)
- **Production URL**: https://ucag-nine.vercel.app
- **Institutions with real data in the codebase**: 9 (UMP, UP, Wits, NWU, NMU, UKZN, TUT, Stellenbosch, UCT) — **1 of 9 (UMP) is actually live in production** as of this document
- **Institutions registered but without a real programme catalogue yet**: UJ (APS rule only, catalogue blocked by source-site 403), CPUT, UNISA (deliberately unseeded, no research done)
- **Real programmes in the codebase**: 138+ across the 9 institutions above (12 of them currently live)
- **Real bursaries/internships in the codebase**: ~37 (1 currently live)
- **AI provider**: Google Gemini, free tier (`gemini-flash-lite-latest`)